

UNIVERSITÀ DEGLI STUDI DI MODENA
E REGGIO EMILIA

**Department of Sciences and Methods for
Engineering**

*Master's Degree Course in Digital Automation Engineering
High Performance Computing for Advanced Physical
Analysis*

OpenMP-Driven Monte Carlo: From Sequential to Hybrid

Candidates:

Ines Malfettone

student ID number: 203487

Federico Fantini

student ID number: 206987

Academic year 2025/2026

Abstract

This project investigates the parallelization of a Monte Carlo integration algorithm using OpenMP, MPI, and a Hybrid MPI–OpenMP approach. Starting from a sequential implementation, three parallel versions were developed and compared in terms of execution time, speedup and efficiency.

Particular attention was given to the generation of pseudo-random numbers, since the initial implementation based on the intrinsic Fortran random number generator was not suitable for the multithreaded versions. For this reason, a custom thread-private linear congruential generator was introduced and shared among all four implementations in order to obtain a consistent execution model.

The experimental analysis was carried out on the Lyra HPC cluster using 10^8 Monte Carlo samples. The results show that both the OpenMP and Hybrid implementations achieve a significant reduction in execution time as the number of processing elements increases. The Hybrid version provides the best performance among the tested configurations, while maintaining numerical results consistent with the sequential solution. The MPI implementation also shows a clear reduction in execution time when increasing the number of processes.

Overall, the project highlights the main differences between shared-memory, distributed-memory, and hybrid parallel programming, and shows how workload distribution, thread management, random number generation, and process/thread affinity can directly affect the scalability of a parallel application.

Declaration on the Use of AI Tools

Artificial Intelligence tools were used in this project only as a supporting resource for learning and debugging. Their use was limited to specific tasks and all suggestions were independently reviewed, adapted, and experimentally verified by the authors.

The main use concerned the debugging of the pseudo-random number generation strategy. The initial implementation of the four programs used the `mc_rng.f90` module, based on the approach adopted in the course material. However, during the experimental tests, the OpenMP and Hybrid versions showed an unexpected lack of scalability. Since the Fortran intrinsic `RANDOM_NUMBER` routine is documented as thread-safe, the origin of the performance problem was not immediately evident. AI was therefore used as an additional debugging tool to investigate the interaction between the random-number generator and concurrent OpenMP threads and to compare possible solutions. The resulting changes were then reviewed and tested directly on the Lyra cluster before being included in the final implementation.

Finally, AI was used to support the preparation of the SLURM job scripts, mainly to manage execution parameters and file paths according to the chosen repository structure, so that executables, program outputs, and SLURM outputs were stored in the intended directories.

Contents

1	Introduction	4
2	Basic Syntax and Compiler Directives	5
2.1	Directives	5
2.1.1	Thread Management and Control	6
2.2	The <code>omp.lib</code> Module and Runtime Procedures	8
3	The Execution Model	9
4	Data Management	10
4.1	Data Sharing Attribute Clauses	11
4.1.1	Shared Variable	11
4.1.2	Private Variable	12
4.1.3	Firstprivate Variable	12
4.1.4	Lastprivate Variable	13
4.1.5	Advanced Data Environment Clauses	14
4.2	The False Sharing Problem	16
5	Comparative Analysis: OpenMP versus MPI	17
6	The Three-Dimensional Monte Carlo Integral	18
6.1	Monte Carlo estimator	19
6.2	Reason for selecting the problem	19
6.3	Correspondence with the repository	19
7	Implementations	20
7.1	Random number generation: from <code>mc_rng</code> to <code>mc_rng1</code>	20
7.2	Repository organization and Slurm job scripts	24
7.3	Sequential reference version	26
7.4	OpenMP version	26
7.5	MPI version	27
7.6	Hybrid MPI+OpenMP version	29
8	Experimental Methodology	32
8.1	Hardware and software environment	32
8.2	Numerical check before benchmarking	33
8.3	Timing methodology	33
8.4	Strong-scaling experiments	34
8.5	Performance metrics	34
9	Results	35
9.1	Single-resource comparison	35
9.2	OpenMP, MPI and hybrid strong-scaling results	36
9.3	Strong-scaling figures	37
9.3.1	OpenMP Results	37
9.3.2	MPI Results	38
9.3.3	Hybrid Results	39
10	Discussion	40
11	Conclusions	41

References

43

1 Introduction

In the field of High Performance Computing (HPC), the demand for massive computing power to solve complex scientific, engineering, and industrial problems has made parallel computing essential. As a result, modern computer processors rely heavily on multi-core architectures, where overall performance is boosted by distributing workloads across multiple processing units that operate simultaneously.

To make the most of these multi-core systems, developers use OpenMP, which is a directive-based API designed for shared-memory parallel programming in C, C++ and Fortran. OpenMP relies on a shared memory model where all processor cores have access to a common, global address space, interacting through shared L3 cache.

From an execution perspective, OpenMP programs generally align with the SPMD (Single Program, Multiple Data) model, where multiple threads execute identical blocks of code concurrently. However, unlike distributed-memory paradigms that enforce strict data isolation, OpenMP provides fluid flexibility in data management. Because all threads share a common address space, they can seamlessly operate on independent private data or collaboratively read and write to shared variables. This behavior is precisely governed by the programmer using explicit scoping clauses, such as *private* and *shared*, enabling threads to either cooperate on a unified dataset or process distinct workloads as required [1].

The shared memory architecture brings both major advantages and unique challenges. Unlike distributed-memory paradigms (such as MPI), where each computing node has its own isolated memory and data transmission requires explicit network calls like *MPI_Send* and *MPI_Recv* [2], shared memory communication happens behind the scenes. If one core writes data to a specific memory address, that value instantly becomes available and readable to any other core pointing to the same location. Because there are no communication bottlenecks caused by external network cables, information exchange is extremely fast. However, this freedom of shared access introduces the primary challenge of the model: if multiple cores try to write to the same variable at the same time without proper coordination, it leads to conflicts known as data races, which can corrupt the program's results.

Beyond hardware execution, OpenMP offers important benefits for software development and code maintenance. Without OpenMP, managing threads manually would require low-level operating system APIs, forcing developers to write hundreds of lines of complex code just to create, synchronize, and destroy threads. OpenMP completely abstracts this complexity, reducing it to simple text-based directives.

Furthermore, from an implementation standpoint, turning sequential code into parallel code does not require a complete rewrite of the software. Developers can take an existing sequential application and parallelize it gradually, focusing exclusively on the heaviest computational bottlenecks, such as heavy loops, without having to redesign the entire software architecture.

Before diving deeper into OpenMP mechanics, it is essential to clarify several fundamental terms that define how parallel execution is structured, managed, and distributed. Let's look at the key concepts:

- **Process:** independent execution unit managed by the operating system, possessing its own completely isolated memory space and system resources. They are heavy weight entities; their creation, destruction, and context switching require significant computational overhead, and

data exchange requires explicit communication mechanisms (such as MPI) due to memory isolation.

- **Thread:** smallest sequence of programmed instructions. Threads are part of a process, meaning multiple threads belong to a single process and share its common memory address space. They are light weight, making their creation, termination, and synchronization extremely fast. Communication happens instantly behind the scenes via shared memory (L3 cache).
- **Team:** refers to the group of threads cooperating to execute a parallel task. When an OpenMP program enters a parallel construct, the system creates or activates a team consisting of a master thread and multiple worker threads working together toward a common goal.
- **Parallel Region:** specific block of code that is executed concurrently by a team of multiple threads.
- **Work Sharing:** mechanism that distributes a task, such as a computational loop, among the members of a thread team, ensuring that each thread processes a distinct portion of the work rather than repeating it entirely.

2 Basic Syntax and Compiler Directives

In this chapter, we will explore the essential prerequisites required to execute an OpenMP program and examine the fundamental syntax needed to write parallel code. We will cover how to structure basic parallel regions, utilize compiler directives, and interact with the OpenMP runtime environment to control thread behavior effectively. Additionally, we will discuss the *fallback mechanism*.

2.1 Directives

A fundamental step is understanding how developers concretely interface with OpenMP. Communication between the source code and the compiler occurs almost entirely through directives. A directive is an instruction addressed exclusively to the compiler, designed to indicate how a specific block of sequential code should be transformed into a parallel one. Regardless of the programming language used, the anatomy of an OpenMP directive follows a common structure composed of three main logical elements:

- **Sentinel:** An initial marker that signals to the compiler that the line belongs to OpenMP and should not be ignored as a standard comment.
- **Directive Name (Keyword):** Specifies the action for the compiler to execute, such as `parallel` to indicate that a code block must be executed concurrently, or `do/for` to distribute loop iterations.
- **Optional Clauses:** Used to modify or add detail to a directive. For instance, `num_threads()` defines the exact number of threads to be created for that parallel execution.

In Fortran the general syntax follows this structure:

```
!$omp directive [clauses]
```

Since Fortran does not use curly braces to delimit code blocks, explicitly closing the parallel region is required using a corresponding end directive (e.g., `!$omp end parallel`). Below is a practical Fortran implementation example that shows how to instruct the compiler to execute a code block in parallel using exactly 4 threads:

Listing 1: OpenMP basic parallel region

```

1 !$omp parallel num_threads(4)
2   ! Code executed in parallel
3 !$omp end parallel

```

As noted earlier, the line starts with an exclamation mark (!), which typically designates the beginning of a comment in Fortran. A natural question arises: how does the compiler distinguish an OpenMP directive from a standard comment? This distinction is controlled by enabling a specific compilation flag, such as `-fopenmp` in `gfortran` or `-qopenmp` in `mpiifort` [7]. When this flag is active, the compiler interprets the directive and generates parallel code; otherwise, it treats the line as a normal comment and compiles the program sequentially. This behavior is known as **fallback mechanism**. While it prevents compilation errors and allows the software to remain functional without parallel support, it can occasionally lead to silent failures, where a developer assumes that the program is running in parallel when it is actually executing sequentially. Discovering this discrepancy often requires analyzing execution times or monitoring CPU utilization. Ultimately, this design ensures that a single codebase remains unified, eliminating the need to maintain separate sequential and parallel versions of the software.

2.1.1 Thread Management and Control

In order to configure and determine the number of threads within a parallel region three primary methods can be used, which operate at different levels of abstraction:

- **Operating System Environment Variable (OMP_NUM_THREADS):** This is the most widely used, cleanest, and recommended approach in High Performance Computing (HPC), as it allows modifying the thread count without requiring code recompilation. From the command line or bash shell, the syntax is:

```
export OMP_NUM_THREADS=4
```

This variable is registered by the OS prior to execution. When the program encounters a parallel region at runtime, it reads this assigned value and dynamically allocates a corresponding number of threads.

- **Runtime Function (omp_set_num_threads()):** This function enables setting the thread count directly inside the source code. Consequently, modifying the number of threads requires altering the code and performing a new compilation. The syntax is:

Listing 2: OpenMP - omp_set_num_threads routine

```

1 call omp_set_num_threads(4)
2 !$omp parallel
3   ! Code executed in parallel by 4 threads
4 !$omp end parallel

```

From this point onward, every subsequent parallel region will be executed by 4 threads, unless the value is explicitly overridden.

- **Local Clause (`num_threads()`):** This is the most specific and localized method, applied directly as a clause within the parallel directive. The syntax is:

Listing 3: OpenMP - `num_threads` clause

```

1  !$omp parallel num_threads(4)
2      ! Code executed in parallel by 4 threads
3  !$omp end parallel

```

This instruction applies exclusively to that specific code block and overrides any previous thread count settings for that region only.

Since these three methods can coexist within the same program, OpenMP enforces a strict hierarchical priority to resolve any potential conflicts. Listed in order from lowest to highest priority, the hierarchy is as follows:

1. **System Default:** Automatic runtime selection based on the underlying hardware (e.g., the number of available logical cores).
2. **Environment Variable (`OMP_NUM_THREADS`):** Overrides the system default value.
3. **Runtime Function (`omp_set_num_threads()`):** Overrides the environment variable for all subsequent parallel regions.
4. **Clause (`num_threads()`):** Holds the absolute highest priority. When applied directly to a directive, it overrides any prior settings (whether derived from runtime functions or environment variables) exclusively for that specific code block [1].

A common requirement when implementing parallel software is the ability to execute a specific code block with a thread count different from other parallel regions. For instance, a developer might allocate a higher number of threads for computationally intensive tasks (such as heavy mathematical modeling) while utilizing fewer threads for lighter phases (such as I/O operations). This is achieved by leveraging the maximum priority of the `num_threads()` clause, applying it individually to parallel regions to override the previous configuration.

A practical example is illustrated below:

Listing 4: OpenMP - Different thread counts across parallel regions

```

1  ! First parallel region: requests 16 threads
2  !$omp parallel num_threads(16)
3      ! [Heavy parallel workload]
4  !$omp end parallel
5
6  ! Sequential or other operations...
7
8  ! Second parallel region: requests 4 threads
9  !$omp parallel num_threads(4)
10     ! [Lighter parallel workload]
11 !$omp end parallel

```

To fully understand how the OpenMP runtime engine handles these dynamic transitions between parallel regions with differing thread counts, we must first examine the underlying execution model, which will be covered in Chapter 3.

2.2 The omp_lib Module and Runtime Procedures

While OpenMP relies primarily on compiler directives to manage parallel regions and work-sharing, it also provides a comprehensive runtime library API. This API allows developers to inspect and modify the execution environment dynamically during runtime, such as querying thread identifiers or adjusting the number of active threads.

In Fortran, interacting with these runtime routines and utilizing OpenMP requires the inclusion of the intrinsic OpenMP module via the declaration:

```
use omp_lib
```

This statement must be placed at the beginning of the program (before `implicit none`) or at the beginning of the subroutine containing the parallelized sections of code.

By including this module, developers gain access to various runtime routines that enable them to query information regarding parallel executions or dynamically adjust parameters, such as controlling the active thread count as discussed previously.

Procedure	Role in the implementation
<code>omp_set_num_threads()</code>	Sets the number of threads to be used for subsequent parallel regions that do not specify a <code>num_threads</code> clause.
<code>omp_get_num_threads()</code>	Returns the number of threads in the team associated with the current parallel region.
<code>omp_get_max_threads()</code>	Returns an upper bound on the number of threads that could be used to form a new team if a parallel construct without a <code>num_threads</code> clause were encountered after execution returns from this routine.
<code>omp_get_thread_num()</code>	Returns the identifier of the calling thread within the current team. The primary thread is assigned identifier zero. The code uses this value exclusively to identify the thread that records the team size, rather than for manual sample partitioning.
<code>omp_get_wtime()</code>	Returns a wall-clock time in seconds. The difference between values measured before and after the parallel region serves to compute the elapsed execution time.

Table 1: OpenMP runtime procedures used by the Fortran implementation.

To better understand how these runtime procedures operate in practice, the following Fortran implementation demonstrates their integration within a parallel program. This code explicitly sets the thread count, measures the execution time using a wall-clock timer, and retrieves thread specific identifiers during execution.

Listing 5: Practical Fortran example demonstrating OpenMP runtime routines.

```

1 program openmp_runtime_example
2   use omp_lib
3   implicit none
4
5   integer :: actual_threads, thread_id
6   real(8) :: start_time, end_time

```

```

7
8      ! Set the desired number of threads
9      call omp_set_num_threads(4)
10
11     ! Measure the start time
12     start_time = omp_get_wtime()
13
14     ! Beginning of the parallel region
15     !$omp parallel
16
17         ! Obtain the ID of the current thread
18         thread_id = omp_get_thread_num()
19
20         if (thread_id == 0) then
21             actual_threads = omp_get_num_threads()
22         end if
23
24     !$omp end parallel
25
26     ! Measure the end time and calculate the elapsed time
27     end_time = omp_get_wtime()
28
29     print *, "Elapsed_time:", end_time - start_time
30
31 end program openmp_runtime_example

```

Every thread evaluates `omp_get_thread_num()`, but only the thread with identifier zero writes `actual_threads`. This avoids multiple threads writing the same diagnostic variable at the same time.

3 The Execution Model

The execution model defines the fundamental rules governing how a program transitions between sequential and parallel phases throughout its lifecycle. Specifically, OpenMP implements this dynamic behavior through the **fork-join** execution model. Conceptually, the lifecycle of the program flows through four distinct stages: the initial **sequential phase**, the **fork phase**, the **parallel execution phase**, and finally the **join phase**.

Every OpenMP program begins its execution in a traditional manner, operating as a sequential process driven by a single thread, known as the **master thread**. When the execution flow encounters a parallelism directive (such as `!$omp parallel` in Fortran), the **fork phase** is triggered, causing the master thread to spawn or activate a team of auxiliary threads.

Once the team is established, the program enters the **parallel execution phase**, where the code is executed concurrently by all active threads within a shared memory address space. Each thread is assigned a unique identifier (thread ID).

Upon reaching the end of the parallel region, explicitly denoted by a termination directive like `!$omp end parallel`, execution momentarily pauses to enforce a synchronization barrier, ensuring that no thread proceeds until the entire team has completed its workload. Then, the **join phase** takes place: the auxiliary threads are deactivated or suspended, and program control reverts exclusively to the master thread, which resumes standard sequential execution [1].

A critical aspect of this lifecycle is the active role of the master thread during the parallel execution phase. Rather than remaining idle while auxiliary threads process the workload, the master thread seamlessly integrates into the newly formed team, typically assuming the identifier zero, and executes the enclosed parallel code alongside the other members.

At the conclusion of the parallel region, during the join phase, a key architectural question arises: what happens to the auxiliary threads? Rather than repeatedly creating and destroying threads, which incurs a heavy computational cost known as **thread overhead**, modern OpenMP runtimes utilize a **thread pool**. When a parallel region ends, the auxiliary threads are typically suspended rather than terminated. They are returned to an internal pool and placed in a waiting state, ready to be quickly awakened for subsequent parallel regions.

Developers and users can control this dynamic runtime behavior by configuring **environment variables** prior to execution. These variables allow external control over how the runtime library manages thread allocation and synchronization. Let's see two examples:

- **OMP_WAIT_POLICY**: This variable controls how waiting threads behave. Setting it to **ACTIVE** tells threads to keep checking the CPU (busy waiting), which reduces start-up time for future parallel regions at the cost of higher power use. On the other hand, setting it to **PASSIVE** makes threads go to sleep and free up the CPU until they are needed again.
- **OMP_DYNAMIC**: This variable controls whether the runtime can change the number of threads during execution. When disabled (the default), the program strictly follows the exact number of threads requested. When enabled, OpenMP can automatically reduce the thread count based on system load, treating the requested number only as an upper limit [1].

Environment variables are set directly in the operating system's shell terminal before launching the compiled program.

Listing 6: Setting an OpenMP environment variable in the terminal prior to execution.

```
1 export OMP_WAIT_POLICY=active
2 ./fortran_program
```

Furthermore, OpenMP handles changes in the number of threads between consecutive parallel regions efficiently. If a program transitions from a region requiring a smaller team (e.g., two threads) to one requiring a larger team (e.g., four threads), the auxiliary threads from the first region are not permanently destroyed during the join phase. Instead, they are suspended and placed into the internal thread pool. When the subsequent region starts, the runtime immediately wakes up the existing suspended threads and spawns only the additional ones needed to meet the new requirement, minimizing performance overhead. Conversely, when transitioning from a larger team to a smaller one, only the required number of threads is utilized for the new block, while the excess threads remain suspended in the pool without being unnecessarily awakened.

4 Data Management

OpenMP operates on a shared memory programming model, meaning all threads within a parallel region share the same logical address space, which corresponds to the system's memory. Because of this centralized memory structure, thread communication in OpenMP is entirely implicit. Unlike distributed memory models (such as MPI) that require explicit message passing between isolated tasks, OpenMP threads exchange data naturally by reading and writing to the same shared variables.

Despite this unified logical view, the underlying hardware relies on a strictly hierarchical cache architecture. While the main memory and the Last Level Cache (typically L3) are physically shared among all processing cores, each individual core operates using its own private, high speed L1 and L2 caches. When threads access shared variables, hardware cache coherence protocols automatically synchronize the data across their private caches. While this hardware intervention keeps the programming model straightforward, it can introduce severe performance penalties, such as *False Sharing*, if threads repeatedly modify adjacent data within the same cache line.

4.1 Data Sharing Attribute Clauses

To guarantee computational correctness and prevent race conditions, OpenMP requires developers to actively manage how variables are accessed through data sharing attribute clauses. These clauses allow the programmer to define the scope and visibility of every variable explicitly. By strategically determining whether a variable should reside in the globally shared memory or be isolated as a private copy within a thread's local execution context, the developer directly controls the physical data flow, ensuring both mathematical accuracy and optimal hardware utilization. These clauses are appended to parallel directives to explicitly define the scope and behavior of variables during parallel execution.

4.1.1 Shared Variable

A shared variable is unique and resides in shared memory, meaning all threads read from and write to the same logical address. If multiple threads must update it concurrently, the programmer must implement synchronization constructs (such as `atomic` or `critical`) to prevent race conditions. Specifically, `atomic` is the preferred, high-performance choice for protecting a single, basic operation (such as an increment). In contrast, `critical` protects an entire block of code by enforcing mutual exclusion, ensuring that only one thread can execute that section at a time.

To illustrate the behavior of a shared variable and the necessity of synchronization, consider the following Fortran example:

Listing 7: Shared variable with atomic synchronization

```
1 integer :: total_sum
2 total_sum = 0
3
4 !$omp parallel shared(total_sum)
5     !$omp atomic
6     total_sum = total_sum + 1
7 !$omp end parallel
8
9 print *, "Final total_sum: ", total_sum
```

In this example, the variable `total_sum` is explicitly declared as `shared`, which means that all generated threads access the exact same logical memory address. Because every active thread attempts to read, increment, and write back to `total_sum` simultaneously, a race condition is imminent. If two threads read the value at the same microsecond, they will compute the same incremented value and overwrite each other's progress, resulting in lost updates. To prevent data corruption, the `!$omp atomic` directive is placed immediately before increment. In this way, the variable is modified by one thread at a time. Consequently, the final printed sum will perfectly match the total number of threads that executed the block.

In the OpenMP implementation developed for this project, the total number of samples is declared as a shared variable:

```
1 !$omp parallel default(none)
2 !$omp& shared(n_samples, actual_threads)
```

The variable `n_samples` is read by all threads but is not modified during the parallel region. The variable `actual_threads` is shared because its value is recorded by the master thread and used after the parallel region.

4.1.2 Private Variable

When a variable is declared **private**, each thread creates a new, independent copy of it in its local memory. Upon entering the parallel region, this local copy is strictly uninitialized: it does not inherit the value of the original variable. Instead, the newly allocated memory space contains arbitrary residual data (often referred to as garbage values). Consequently, attempting to read the private variable before explicitly assigning it a new value within the parallel region leads to undefined behavior. When the block terminates, the local copies are destroyed, and the value of the original variable outside the block remains completely unchanged.

To better understand this mechanism, consider the following example:

Listing 8: Private variable

```
1 integer :: x
2 x = 100
3
4 !$omp parallel private(x)
5     x = omp_get_thread_num()
6     print *, "Thread local x: ", x
7 !$omp end parallel
8
9 print *, "Global x after parallel region: ", x
```

In this example, the global variable `x` is initialized to 100. When the parallel region begins, the **private(x)** clause directs the runtime to allocate a new, independent instance of `x` for each active thread. If a thread were to read `x` immediately upon entering the block, it would read a random residual value rather than 100. To ensure computational correctness, each thread explicitly initializes its local copy by assigning it its unique thread ID (`omp_get_thread_num()`) before executing the print statement. After the parallel execution concludes, all private copies are safely discarded, and the final print statement outputs 100, proving that the original memory address was never accessed or modified by the active threads.

The same data-sharing strategy is used in `mc_openmp.f90`. The variables associated with the current sample are private to each thread:

```
1 !$omp& private(thread_id, x, y, z, value)
```

Each thread therefore owns its own coordinates and integrand value.

4.1.3 Firstprivate Variable

A **firstprivate** variable is similar to **private**, but eliminates the uninitialized value issue. OpenMP automatically copies the value the variable held immediately before entering the parallel block and assigns it as the initial value to each thread's private copy.

To highlight the contrast with a standard `private` declaration, the following example shows how `firstprivate` loads the initial value:

Listing 9: Firstprivate variable

```

1 integer :: offset
2 offset = 50
3
4 !$omp parallel firstprivate(offset)
5     offset = offset + omp_get_thread_num()
6     print *, "Thread local offset: ", offset
7 !$omp end parallel
8
9 print *, "Global offset after parallel region: ", offset

```

In this code, the global variable `offset` is initialized to 50 prior to the parallel block. Unlike the standard `private` clause, which would leave the local copies containing arbitrary garbage data, `firstprivate` guarantees that every thread begins execution with its private copy of `offset` correctly initialized to 50. Each thread then safely adds its unique thread ID to this baseline value and prints the result. Because these calculations and updates are performed strictly on the isolated local copies, the original global variable is completely unaffected, and the final print statement will output 50.

4.1.4 Lastprivate Variable

The `lastprivate` clause extends the behavior of a standard `private` declaration by guaranteeing the persistence of the result after the computation. At the end of execution, OpenMP copies the value calculated by the *logically final* iteration (the last step of the loop in sequential order) or from the last lexical section into the original external variable. The selection criterion is logical rather than chronological. This clause can only be used within worksharing constructs, such as distributed loops (`!$omp do`) and section blocks (`!$omp sections`). It cannot be applied directly to a generic parallel region.

The following listing shows an example.

Listing 10: Lastprivate variable

```

1 integer :: i, final_val
2 final_val = 0
3
4 !$omp parallel
5 !$omp do lastprivate(final_val)
6 do i = 1, 10
7     final_val = i * 2
8 end do
9 !$omp end do
10 !$omp end parallel
11
12 print *, "Global final_val after loop: ", final_val

```

In this example, the `!$omp do` directive distributes the 10 iterations of the loop across the available threads. Because `final_val` is declared as `lastprivate`, each thread creates its own isolated local copy to compute `i * 2` without memory conflicts. When the loop completes, OpenMP identifies the thread that executed the logically final iteration (where `i = 10`). The value calculated during that specific iteration (`10 * 2 = 20`) is automatically copied back into the original, global

`final_val` variable. Even if a thread executing an earlier iteration (such as `i = 5`) finishes its workload chronologically later, the runtime guarantees that the global variable will safely retain the value 20, preserving the sequential logic of the program [1].

4.1.5 Advanced Data Environment Clauses

While standard data sharing attributes like `shared` and `private` strictly define the memory layout of individual variables, OpenMP also provides advanced mechanisms to manage the broader execution context. This section explores two specialized constructs: the `default` clause, which governs the compiler’s implicit scoping rules to enforce safer programming practices, and the `reduction` clause, a powerful hybrid tool that seamlessly combines memory isolation, local computation, and automated thread synchronization to safely aggregate parallel results.

The `default(shared | none)` clause determines the implicit scoping rules for any variable within a parallel region that lacks an explicit data sharing directive.

Employing `default(none)` is widely considered a best practice: it forces the compiler to issue an error if any variable is left unassigned, thereby eliminating ambiguity and preventing silent concurrency bugs.

To illustrate why `default(none)` is highly recommended, consider a scenario where an array is scaled by a constant factor:

Listing 11: Enforcing explicit data scoping with `default(none)`

```

1 integer :: i, n
2 real :: factor
3 real, dimension(100) :: array
4
5 n = 100
6 factor = 2.5
7
8 ! Sequential initialization of the array
9 do i = 1, n
10     array(i) = real(i)
11 end do
12
13 !$omp parallel do default(none) shared(n, factor, array) private(i)
14 do i = 1, n
15     array(i) = array(i) * factor
16 end do
17 !$omp end parallel do

```

In this example, `default(none)` strips away all implicit OpenMP assumptions. The compiler is forced to verify that every single variable used within the parallel region (`n`, `factor`, `array`, and `i`) has an explicitly defined data sharing attribute.

If the programmer accidentally forgets to include a variable in the clauses (for example, omitting `private(i)` or `shared(factor)`), the compilation will immediately halt with an error. Without `default(none)`, OpenMP might fall back on its implicit rules, such as assuming undeclared variables are `shared`, which can easily introduce silent race conditions if a temporary scalar variable is mishandled. By demanding absolute explicit scoping, `default(none)` acts as a strict safety net for the developer.

The project uses `default(none)` in both the OpenMP and hybrid implementations. This choice

makes the data-sharing decisions explicit:

```

1 !omp parallel default(none) &
2 !omp& shared(n_samples, actual_threads) &
3 !omp& private(thread_id, x, y, z, value) &
4 !omp& reduction(+:sum_value)

```

While the **default** clause focuses on scoping safety, the **reduction** clause is designed specifically for accumulation operations. It generates private copies for each thread, initialized to the chosen operator's identity value (e.g., 0 for addition, 1 for multiplication). Each thread computes its local accumulation in isolation, avoiding memory conflicts. At the end of the block, the runtime safely combines all partial results to update the original shared variable.

The **reduction** clause can be used in a worksharing construct or in a generic parallel region:

Case 1: Reduction with Worksharing (Distributed Loop)

When combined with a worksharing directive like **!omp do**, the iterations of a loop are partitioned among the threads. Each thread computes a partial result for its specific chunk of iterations.

Listing 12: Reduction combined with worksharing

```

1 integer :: i, total_sum
2 total_sum = 0
3 !omp parallel do reduction(+:total_sum)
4 do i = 1, 100
5     total_sum = total_sum + i
6 end do
7 !omp end parallel do
8
9 print *, "Final sum is: ", total_sum ! Outputs 5050

```

In this scenario, the workload is actively divided. If executed with 4 threads, each thread processes 25 iterations. The runtime then safely adds the four partial sums together, guaranteeing that the final output is mathematically identical to sequential execution (5050).

This exact pattern is adopted in the Monte Carlo simulation (`mc_openmp.f90`, fully listed in Listing 24). In the application's implementation, the parallel region combines the global parallel team with a distributed worksharing loop, where the **reduction(+:sum_value)** clause ensures that thread-local accumulations are safely merged into the global variable without race conditions.

Case 2: Reduction in a Generic Parallel Region

If **reduction** is applied directly to an **!omp parallel** region without any worksharing loop, every thread executes the entire block of code independently. This pattern is conceptually identical to **MPI_Reduce** in distributed memory programming: each parallel entity (thread) computes a complete, independent local result, and the runtime aggregates these independent contributions into a single global value.

Listing 13: Reduction on a generic parallel region

```

1 integer :: sum_value
2 sum_value = 0
3 !omp parallel reduction(+:sum_value)
4     sum_value = sum_value + 10 * omp_get_thread_num()
5 !omp end parallel
6
7 print *, "Aggregated sum from all threads: ", sum_value

```

Here, no work is automatically distributed. The **reduction** mechanism creates a private, isolated copy of **sum** for each thread, automatically initialized to 0 (the identity value for addition). If executed by 4 threads (0, 1, 2, and 3), they independently evaluate the expression and overwrite their local copies with the values 0, 10, 20, and 30. Because each thread writes exclusively to its own memory space, no race conditions occur. At the **!\$omp end parallel** barrier, the OpenMP runtime automatically synchronizes the threads and safely merges these private copies to update the original global variable, producing a final **sum** of 60 [1].

This approach is highly effective when threads operate on independent data streams or execute local simulations before a final aggregation phase.

4.2 The False Sharing Problem

In shared memory parallel programming, performance can be severely degraded by a subtle hardware level issue known as *False Sharing*. Modern CPUs do not fetch memory byte-by-byte; instead, they load data in fixed size blocks called *cache lines* (typically 64 bytes).

False sharing occurs when two or more threads, running on different processor cores, modify independent variables that happen to reside on the exact same cache line [6]. Even though the variables are logically unrelated, the hardware's cache coherence protocol manages synchronization of entire cache lines rather than individual variables.

A classic example of this vulnerability involves using a shared array to store thread specific partial results:

Listing 14: False sharing example

```

1 integer :: partial_sums(4)
2 partial_sums = 0
3
4 !$omp parallel shared(partial_sums)
5     ! Threads write to contiguous memory locations simultaneously
6     partial_sums(omp_get_thread_num() + 1) = omp_get_thread_num() * 10
7 !$omp end parallel

```

In this code, elements like **partial_sums(1)** and **partial_sums(2)** are stored contiguously in memory and therefore share the same cache line. When Thread 0 and Thread 1 update their respective indices simultaneously, they trigger a devastating hardware phenomenon known as the **ping-pong effect**.

The Hardware Ping-Pong Mechanism

When independent variables are packed too closely together within the same cache line, execution degrades into a continuous loop of cache invalidations and hardware stalls:

1. **Initial Modification:** Thread 0 modifies its variable (e.g., **partial_sums(1)**). The CPU loads the entire cache line, containing the contiguous array elements, into Thread 0's L1 cache, and the hardware coherence circuit marks the block as modified.
2. **Cross-Invalidation:** The copy of that specific cache line residing in Thread 1's L1 cache is instantly marked as obsolete and unusable.
3. **Hardware Stall:** When Thread 1 attempts to update its adjacent variable (**partial_sums(2)**), it encounters a cache miss because its local cache line has been invalidated, forcing a core execution stall.

4. **Reload and Write:** Thread 1 is forced to slowly re-fetch the entire cache line from shared memory (such as L3) into its L1 cache before applying its modification.
5. **Role Reversal and Repetition:** Thread 1's write triggers the coherence protocol once again, invalidating the cache line back in Thread 0's cache.

Consequently, the processor cores spend their time fighting over memory ownership and reloading data rather than executing actual computations. This results in a massive increase in memory traffic and a drastic drop in execution speed.

Mitigation Strategies

To prevent false sharing and restore high performance, several standard strategies can be employed:

- **Local Private Accumulators:** Utilizing temporary local variables (stored in thread private registers) for heavy intermediate calculations, writing to shared data structures only at the very end of execution.
- **Native OpenMP Constructs:** Leveraging explicit clauses like `private` and `reduction`, which automatically allocate temporary variables in local memory. This physical separation in hardware naturally bypasses false sharing and maximizes parallel efficiency.
- **Memory Padding:** Artificially spacing out shared data structures (e.g., using specific array or dummy variables) to mathematically guarantee that independent variables reside on completely separate cache lines.

In that project, the hybrid implementation applies the same OpenMP data management strategy within each MPI process. The local sampling loop uses private variables and an OpenMP reduction on `local_sum`. After all OpenMP threads have completed their local computation, each MPI process participates in `MPI_Reduce`, which combines the local sums and returns the global sum to rank zero.

5 Comparative Analysis: OpenMP versus MPI

The parallelization of scientific applications fundamentally relies on two distinct hardware paradigms: shared memory and distributed memory architectures. To address these environments, two dominant programming models have emerged: OpenMP for shared memory systems and the Message Passing Interface (MPI) for distributed infrastructures.

The main difference between OpenMP and MPI comes down to how they handle memory. OpenMP is designed for a single multi-core computer, where all threads share the same memory space. Threads can communicate easily by reading and writing to common variables, though developers must use synchronization tools to avoid race conditions. In contrast, MPI is built for distributed systems where multiple independent computers are connected over a network. Each MPI process has its own private memory and cannot access other processes data directly. To share information, processes must explicitly send and receive messages using communication commands such as `MPI_Send`, `MPI_Recv`, or `MPI_Reduce` [1, 2].

Aspect	OpenMP	MPI
Programming paradigm	Shared-memory parallelism.	Message-passing between processes.
Execution entities	Threads belonging to a single process, identified by a thread ID.	Multiple processes, each identified by a unique rank in a communicator.
Memory model	Unified address space: threads access shared variables, with options for private scopes.	Completely private address spaces: data cannot be accessed remotely and must be exchanged explicitly.
Data distribution	Work-sharing constructs automatically partition loop iterations among threads.	Workloads and data chunks are explicitly partitioned across ranks.
Communication	Implicit via shared memory reads and writes. Relies on proper synchronization.	Explicit via message-passing procedures (e.g., <code>MPI_Send</code> , <code>MPI_Bcast</code> , <code>MPI_Reduce</code>).
Synchronization	Implicit barriers (e.g., at the end of loops) and explicit constructs when needed.	Collective participation, explicit barriers, and communication completion semantics.
Main overheads	Thread scheduling, synchronization barriers, reductions, and memory contention (e.g., false sharing).	Process startup, network latency and bandwidth, collective operations, and load imbalance.
Programming complexity	Focuses on data scoping rules, preventing data races, and avoiding false sharing.	Focuses on data ownership, rank coordination and communication order.
Typical use in this project	Parallelizing independent loops and performing local thread reductions.	Distributing workloads across nodes and combining results globally.

Table 2: OpenMP and MPI

6 The Three-Dimensional Monte Carlo Integral

The practical problem selected for this project is the numerical evaluation of the following three-dimensional integral:

$$I = \int_0^1 \int_0^1 \int_0^1 e^{-(x^2+y^2+z^2)} dx dy dz. \quad (1)$$

The integration domain is the unit cube $[0, 1]^3$. The integrand is:

$$f(x, y, z) = e^{-(x^2+y^2+z^2)}. \quad (2)$$

The sequential, OpenMP, MPI and hybrid programs solve the same mathematical problem. The domain, integrand, sample count and floating-point precision are kept unchanged so that the four programming models can be compared fairly.

6.1 Monte Carlo estimator

Let (x_i, y_i, z_i) be independent samples uniformly distributed in the unit cube. Since the volume of the domain is one, the Monte Carlo estimate is the sample average

$$\hat{I}_N = \frac{1}{N} \sum_{i=1}^N f(x_i, y_i, z_i). \quad (3)$$

Each iteration of the sampling loop performs three operations:

1. generate three uniformly distributed coordinates;
2. evaluate the exponential integrand;
3. add the value to a partial sum.

Different samples are independent [2]. The value computed for sample i is not needed to generate or evaluate sample $i + 1$. The only global operation is the combination of the partial sums. OpenMP performs this operation through a reduction, while MPI uses a collective reduction.

The sample count is read from the fixed input file and is used by all four programs. It remains unchanged when the estimates and execution times are compared and during the OpenMP, MPI and hybrid strong-scaling experiments. Therefore, the performance study changes the execution model and the amount of parallelism, but not the mathematical problem.

6.2 Reason for selecting the problem

The problem is connected with the Monte Carlo material presented in the course and with the course examples based on local sample counts and global summation. It is suitable for this OpenMP project for four main reasons:

- the main loop contains a large number of independent iterations;
- an OpenMP work-sharing construct can distribute the samples among threads without changing the estimator;
- the MPI version requires only sample-count decomposition and a reduction of the local sums;
- the hybrid version uses the same MPI decomposition and applies OpenMP to the loop executed by each process.

6.3 Correspondence with the repository

The six Fortran source files are available in the **online project repository** and are stored in its root directory.

The four executable programs are:

- `mc_sequential.f90`, the reference implementation;
- `mc_openmp.f90`, the shared-memory implementation;
- `mc_mpi.f90`, the distributed-memory implementation;
- `mc_hybrid.f90`, the MPI+OpenMP implementation.

The common input routine is contained in `mc_io.f90`, while the final random number generator used by all four versions is contained in `mc_rng1.f90`. Every program reads the total number of samples from `input/input.dat`. Each implementation writes to its own output file:

- `output/output_sequential.dat` for the sequential version;
- `output/output_openmp.dat` for the OpenMP version;
- `output/output_mpi.dat` for the MPI version;
- `output/output_hybrid.dat` for the hybrid version.

The `job/` directory contains the four Slurm scripts used to compile and launch the programs on Lyra, while `output-job/` contains the Slurm standard-output files. Repeated executions of the same implementation replace its corresponding `output/output_*.dat` file, whereas the Slurm logs remain distinct because the job identifier is included in their filename.

7 Implementations

The four programs compute the same Monte Carlo estimator and use the same file interface. Input handling is shared through `mc_io.f90` and all implementations use the same random number module, `mc_rng1.f90`. The exponential integrand is evaluated directly inside each sampling loop.

Although the four programs use different parallelization strategies, they follow the same computational sequence. First, the total number of Monte Carlo samples is read from the input file. Then the random-number generator is initialized, the samples are generated and accumulated, and finally the estimate is obtained by dividing the global sum by the total number of samples. The main difference between the implementations is therefore how the sampling loop and its partial sum are distributed among the available processing elements. The following subsections follow this execution order so that the differences between the four versions can be compared directly.

7.1 Random number generation: from `mc_rng` to `mc_rng1`

The first version of the project used a module called `mc_rng`. Its purpose was to provide a small common interface around the Fortran intrinsic procedures `RANDOM_SEED` and `RANDOM_NUMBER`. The seed routine first queried the size of the intrinsic seed vector, allocated the vector and combined the system clock with a user-supplied value:

Listing 15: Seed generation used in the first `mc_rng` module.

```

1 module mc_rng
2   use iso_fortran_env, only : real64
3   implicit none
4   private
5   integer, parameter :: seed_spacing = 37
6   public :: rng_seed, rng_uniform
7
8 contains
9   subroutine rng_seed(seed_value)
10
11     integer, intent(in) :: seed_value
12     integer :: seed_size
13     integer :: i
14     integer, allocatable :: seed(:)
15     integer :: clock_time

```

```

16
17     call random_seed(size=seed_size)
18     allocate(seed(seed_size))
19
20     call system_clock(count=clock_time)
21     seed = clock_time + seed_spacing * (/ (i+seed_value, i=1, seed_size
22         ) /)
23
24     call random_seed(put=seed)
25     deallocate(seed)
26 end subroutine rng_seed
...

```

The generation routine itself was very small. Each call returned the three coordinates required by one Monte Carlo sample:

Listing 16: Random point generation in the original `mc_rng` module.

```

26 ...
27 function rng_uniform() result(values)
28
29     real(real64) :: values(3)
30     call random_number(values)
31
32 end function rng_uniform
33
34 end module mc_rng

```

Therefore, the original sampling loop did not directly interact with `RANDOM_NUMBER`: it only called the common `rng_uniform` interface. This allowed the four implementations to share the same RNG module while keeping the Monte Carlo computation independent from the generator internals.

In the sequential version the interface was used by calling `rng_seed` once before the loop and then requesting one three-dimensional random point at each iteration. The same public interface was initially intended for the parallel versions, using thread or rank dependent values as the argument of `rng_seed`.

This design became problematic when OpenMP threads were introduced. The state of the intrinsic Fortran generator is maintained inside the compiler runtime, so it is not represented by an application variable that can be declared `private` or `threadprivate`. Although the intrinsic routine is implemented as thread-safe by the compiler runtime used on Lyra [3], thread safety does not imply that every thread owns an explicitly independent generator state or that simultaneous calls scale efficiently. During the first experiments, the threaded and hybrid versions showed a strong loss of scalability. This indicated that concurrent access to the intrinsic generator introduced contention or runtime serialization inside a loop that should otherwise be fully independent [4].

For this reason the final version replaces the intrinsic generator with an explicit linear congruential generator (LCG) implemented in `mc_rng1.f90`. The module exposes only the two procedures that are required by the four Monte Carlo programs, namely `rng_seed` and `rng_uniform`, while the internal state and the function used to update it remain private.

Listing 17: Definition of the final `mc_rng1` module and its explicit RNG state.

```

1 module mc_rng1
2     use iso_fortran_env, only : real64, int64

```

```

3  implicit none
4  private
5  public :: rng_seed, rng_uniform
6  integer(int64) :: rng_state
7  !$omp threadprivate(rng_state)
8  ...

```

The generator state is stored in the 64-bit integer `rng_state`. This is the main difference with respect to the original version based on `RANDOM.NUMBER`, where the internal state was managed by the Fortran runtime and was therefore not directly accessible from the application.

The OpenMP `threadprivate` directive gives every OpenMP thread its own copy of `rng_state`. Consequently, two threads do not update the same state while generating random numbers [1]. In a pure MPI execution this mechanism is not required for process separation, since each MPI rank already has its own address space and therefore naturally owns an independent copy of the module variable.

After the module declarations, the first procedure defined in `mc_rng1.f90` is `rng_seed`. Its purpose is to initialize the state associated with the current execution context:

Listing 18: Initialization of the final per-thread/per-process RNG state.

```

8  ...
9  contains
10  subroutine rng_seed(seed_value)
11
12      integer, intent(in) :: seed_value
13      integer :: clock_time
14
15      call system_clock(count=clock_time)
16
17      rng_state = modulo( &
18          int(clock_time, int64) + int(seed_value, int64) * 987654321
19          _int64, &
20          2147483648_int64)
21  end subroutine rng_seed
22  ...

```

The initialization combines two values. The first one, `clock_time`, is obtained from `system_clock` and changes between different program executions. The second one, `seed_value`, identifies the process or thread that is requesting its RNG state. The factor 987654321 is not a parameter of the LCG itself: it is used only during initialization as a spacing factor, so that different values of `seed_value` are mapped to sufficiently different initial states.

The final `modulo` operation restricts the initial state to the interval used by the generator:

$$0 \leq \text{rng_state} < 2^{31} = 214783648.$$

This operation is also important for integer safety. Since the state is reduced below 2^{31} before the first update, the multiplication performed later by the LCG remains inside the range of a signed 64-bit integer. This is particularly important in the hybrid implementation, where the seed identifier contains both the MPI rank and the OpenMP thread ID.

The four programs call the same `rng_seed` routine but provide different identifiers:

Implementation	Call to <code>rng_seed</code>
Sequential	<code>rng_seed(base_seed)</code> , with <code>base_seed = 12345</code>
OpenMP	<code>rng_seed(thread_id)</code>
MPI	<code>rng_seed(rank)</code>
Hybrid	<code>rng_seed(rank * 1000 + thread_id)</code>

Table 3: Seed identifier passed to `mc_rng1` by each implementation.

In the sequential version only one generator state is required, so the fixed identifier `base_seed = 12345` is used. In the OpenMP implementation, each thread passes its own `thread_id`, while in the MPI implementation each process uses its `rank`. The Hybrid version must distinguish both levels of parallelism and therefore uses

$$\text{rank} \times 1000 + \text{thread_id}.$$

The factor 1000 is a simple spacing convention. For example, thread 3 of rank 0 receives the identifier 3, while thread 3 of rank 1 receives 1003. Assuming fewer than 1000 threads per MPI process, two different rank/thread pairs cannot produce the same identifier through this expression.

Since `system_clock` also contributes to the initialization, the current implementation is not intended to reproduce exactly the same random sequence across separate executions of the program.

The second procedure defined in the module is `rng_uniform`. This is the routine actually called inside the Monte Carlo loops of the four implementations:

Listing 19: Generation of the three Monte Carlo coordinates in `mc_rng1`

```

21 ...
22   subroutine rng_uniform(x, y, z)
23     real(real64), intent(out) :: x, y, z
24
25     x = get_random()
26     y = get_random()
27     z = get_random()
28   end subroutine rng_uniform
29 ...

```

Each Monte Carlo sample requires three independent coordinates in the unit interval. For this reason, `rng_uniform` calls the internal `get_random` function three consecutive times and assigns the returned values to `x`, `y` and `z`. The Monte Carlo programs therefore do not need to know how the generator is implemented: they only call

```

1 call rng_uniform(x, y, z)

```

and receive the three coordinates required to evaluate the integrand.

The actual update of the generator state is performed by `get_random`, which is the last procedure defined in the module. It is private and can therefore only be called internally by `mc_rng1`:

Listing 20: LCG update and conversion to a real number in $[0, 1)$

```

29 ...
30   function get_random() result(r)
31     real(real64) :: r

```

```

32     rng_state = modulo(rng_state * 1103515245_int64 + 12345_int64, &
33                        2147483648_int64)
34
35     r = real(rng_state, real64) / 2147483648.0_real64
36 end function get_random
37
38 end module mc_rng1

```

The state update follows the linear congruential relation

$$X_{n+1} = (1103515245 X_n + 12345) \bmod 2^{31}. \quad (4)$$

In Equation 4, X_n corresponds to the current value of `rng_state`, while X_{n+1} is the state produced for the next random number. The constants used in the implementation are therefore:

- 1103515245: the multiplier;
- 12345: the increment;
- $2147483648 = 2^{31}$: the modulus.

These values define the 31-bit linear congruential generator adopted in the project [5]. After the integer state has been updated, it is converted to `real64` and divided by 2^{31} :

$$r = \frac{X_{n+1}}{2^{31}}.$$

Since the state satisfies

$$0 \leq X_{n+1} < 2^{31},$$

the resulting value satisfies

$$0 \leq r < 1.$$

Therefore, every invocation of `get_random` produces one double-precision pseudo-random value in $[0, 1)$, and one call to `rng_uniform` advances the LCG state three times to obtain the complete three-dimensional Monte Carlo point.

7.2 Repository organization and Slurm job scripts

The online project repository is organized as follows:

Listing 21: Structure of the code repository.

```

1 HPC-Fantini-Malfettone-Project/
2 |-- input/
3 |   |-- input.dat
4 |-- output/
5 |-- output-job/
6 |-- job/
7 |   |-- job_sequential.sh
8 |   |-- job_openmp.sh
9 |   |-- job_mpi.sh
10 |   |-- job_hybrid.sh
11 |-- mc_sequential.f90
12 |-- mc_openmp.f90
13 |-- mc_mpi.f90

```

```

14 |-- mc_hybrid.f90
15 |-- mc_rng1.f90
16 |-- mc_io.f90
17 |-- mc_rng.f90
18 |-- results_N_10_8.xlsx
19 |-- HPC_Project.pdf

```

The file `results_N_10_8.xlsx` contains the strong scaling analysis with $N = 10^8$ samples.

The job scripts are launched from the `job/` directory. Each script then executes

```

1 cd "$SLURM_SUBMIT_DIR/.."

```

so compilation and execution take place in the project root. As a consequence, the executables `mc_sequential`, `mc_openmp`, `mc_mpi` and `mc_hybrid` are created next to the Fortran source files.

The four scripts share the same Lyra account and partition, request one node, load the Slurm and Intel oneAPI modules, compile with `mpiifort -O2` and write the Slurm log to `output-job/`. The latter directory must already exist when `sbatch` is issued because Slurm opens the standard-output path before the commands in the script are executed. The job identifier `%j` is included in the log filename, so different submissions do not overwrite one another.

The relevant differences between the jobs are:

- **Sequential:** one Slurm task and one CPU are requested. The program is compiled without OpenMP and launched directly with `./mc_sequential`.
- **OpenMP:** one Slurm task is used and `--cpus-per-task` selects the number of threads. The source is compiled with `-qopenmp`. The script defines the OpenMP places at core granularity with `OMP_PLACES=cores` and applies the `close` thread-binding policy through `OMP_PROC_BIND=close`.
- **MPI:** `--ntasks` selects the number of MPI processes. The program is started with `mpiexec -n "$SLURM_NTASKS" ./mc_mpi`. No OpenMP compiler flag is required.
- **Hybrid:** `--ntasks` gives the number of MPI processes and `--cpus-per-task` gives the number of OpenMP threads per process. The code is compiled with `-qopenmp`; the OpenMP affinity variables are the same as in the pure OpenMP job. In addition, `I_MPI_PIN_DOMAIN=omp` is set so that the Intel MPI pinning domain assigned to each rank is compatible with the OpenMP team inside that rank.

The hybrid job makes this two-level resource allocation particularly explicit. For example, the default configuration uses four MPI ranks with five OpenMP threads per rank:

Listing 22: Relevant resource and affinity settings of the Hybrid Slurm job.

```

1 #SBATCH --ntasks=4
2 #SBATCH --cpus-per-task=5
3
4 export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
5 export OMP_DYNAMIC=FALSE
6 export OMP_PLACES=cores
7 export OMP_PROC_BIND=close
8 export I_MPI_PIN_DOMAIN=omp
9
10 mpiifort -O2 -qopenmp mc_io.f90 mc_rng1.f90 mc_hybrid.f90 -o mc_hybrid
11 mpiexec -n "$SLURM_NTASKS" ./mc_hybrid

```

In this example Slurm allocates four MPI processes and five CPUs to each process. `OMP_NUM_THREADS` then creates five OpenMP threads inside each rank, for a total of $4 \times 5 = 20$ processing elements.

7.3 Sequential reference version

The sequential program initializes `mc_rng1` once with the fixed application seed identifier `base_seed = 12345`. The system clock is still mixed into the actual RNG state by `rng_seed`. The program then generates N three-dimensional samples and accumulates the result in double precision.

Listing 23: Main computational flow of the sequential implementation.

```

22 ...
23 call read_sample_count(n_samples, io_status)
24 ...
25 call rng_seed(base_seed)
26
27 sum_value = 0.0_real64
28
29 call cpu_time(start_time)
30
31 do sample = 1_int64, n_samples
32     call rng_uniform(x, y, z)
33
34     value = exp(-(x*x + y*y + z*z))
35     sum_value = sum_value + value
36 end do
37
38 call cpu_time(end_time)
39
40 estimate = sum_value / real(n_samples, real64)
41 ...

```

The sequence in Listing 23 is the reference execution flow used by all the other versions. After reading N from `input/input.dat`, the RNG is initialized once using `base_seed`. The loop then generates one point (x, y, z) at every iteration, evaluates

$$e^{-(x^2+y^2+z^2)},$$

and accumulates it in `sum_value`. Once all N samples have been processed, the Monte Carlo estimate is obtained as `sum_value / n_samples`. Only the sampling loop is included between the two calls to `CPU_TIME`, so input handling, seed initialization, estimate formation and output are excluded from the measured interval.

7.4 OpenMP version

The OpenMP implementation uses a single parallel region to distribute the Monte Carlo sampling loop among a team of threads:

Listing 24: OpenMP parallel region in the final implementation.

```

37 ...
38 !$omp parallel default(none) &
39 !$omp& shared(n_samples, actual_threads) &
40 !$omp& private(thread_id, x, y, z, value) &
41 !$omp& reduction(+:sum_value)
42

```

```

43     thread_id = omp_get_thread_num()
44     call rng_seed(thread_id)
45
46     if (thread_id == 0) then
47         actual_threads = omp_get_num_threads()
48     end if
49
50     !$omp do schedule(static)
51     do sample = 1_int64, n_samples
52         call rng_uniform(x, y, z)
53         value = exp(-(x*x + y*y + z*z))
54         sum_value = sum_value + value
55     end do
56     !$omp end do
57
58     !$omp end parallel
59     ...

```

The `default(none)` clause requires explicit data-sharing attributes for variables without pre-determined attributes. The sample count `n_samples` and the diagnostic variable `actual_threads` are shared, whereas `thread_id`, the coordinates, and the integrand value are private. The loop index `sample` is private by OpenMP rules and therefore does not need to appear in the `private` clause. The `reduction(+:sum_value)` clause gives each thread a private accumulator initialized to zero and combines the partial sums with the original accumulator when the parallel region ends.

Before entering the loop, each thread obtains its identifier and calls `rng_seed(thread_id)` to initialize its own `threadprivate` RNG state. This initialization explains the use of separate `parallel` and `do` directives: each thread must perform these operations before processing its assigned samples. Thread 0 also records the actual team size through `omp_get_num_threads()`; this shared value is read after the parallel region has completed.

The `omp do` directive distributes the loop iterations using `schedule(static)`. Without an explicit chunk size, the iteration space is divided into approximately equal contiguous chunks, with at most one chunk assigned to each thread. The exact chunk sizes and handling of any remainder are implementation-dependent. Since Monte Carlo iterations perform the same sequence of operations and have similar computational costs, static scheduling provides an approximately balanced workload with low scheduling overhead.

The timer starts immediately before the parallel region and stops after its completion. The measured interval includes parallel region entry and exit overhead, thread RNG initialization, the sampling loop, the reduction, and the implicit synchronization at the end of the work-sharing loop and parallel region. Estimate formation and file output are outside the timed interval. The recorded team size is written to `output/output_omp.dat`.

7.5 MPI version

The MPI execution begins by creating the processes and assigning each one a rank. Only rank zero accesses the input file; afterwards the total sample count is distributed to every process through a broadcast:

Listing 25: MPI initialization and distribution of the global sample count.

```

22     ...
23     call MPI_Init(ierr)

```

```

24  call MPI_Comm_rank(MPI_COMM_WORLD, rank, ierr)
25  call MPI_Comm_size(MPI_COMM_WORLD, nprocs, ierr)
26
27  n_total = 0_int64
28
29  if (rank == 0) then
30      call read_sample_count(n_total, io_status)
31      ...
32  end if
33
34  call MPI_Bcast(n_total, 1, MPI_INTEGER8, 0, MPI_COMM_WORLD, ierr)
35  ...

```

After the broadcast, all ranks know both the global problem size `n_total` and the total number of MPI processes `nprocs`. The global iteration space can therefore be partitioned locally by every rank, without any additional point-to-point communication.

Each rank computes its local sample count using

$$n_{\text{base}} = \left\lfloor \frac{N}{P} \right\rfloor, \quad r = N \bmod P, \quad n_p = n_{\text{base}} + \begin{cases} 1, & p < r, \\ 0, & p \geq r. \end{cases} \quad (5)$$

In Equation 5:

- N is the total number of Monte Carlo samples (`n_total`);
- P is the total number of MPI processes (`nprocs`);
- p is the rank of the current MPI process (`rank`), with $0 \leq p < P$;
- $n_{\text{base}} = \lfloor N/P \rfloor$ is the minimum number of samples assigned to every process (`base_points`);
- $r = N \bmod P$ is the number of samples remaining after the integer division (`remainder`);
- n_p is the final number of samples assigned to process p (`n_local`).

The remainder is assigned to the first ranks, so the number of samples differs by at most one. The mathematical decomposition is translated almost directly into the Fortran code:

Listing 26: Quotient and remainder decomposition of the MPI workload

```

40  ...
41  base_points = n_total / int(nprocs, int64)
42  remainder = mod(n_total, int(nprocs, int64))
43
44  n_local = base_points
45
46  if (int(rank, int64) < remainder) then
47      n_local = n_local + 1_int64
48  end if
49  ...

```

For example, if $N = 10$ samples are executed with $P = 3$ processes,

$$n_{\text{base}} = \left\lfloor \frac{10}{3} \right\rfloor = 3, \quad r = 10 \bmod 3 = 1.$$

Therefore, rank 0 receives four samples, while ranks 1 and 2 receive three samples each. The complete workload is still $4 + 3 + 3 = 10$, and the difference between the largest and smallest local workloads is only one sample.

Each rank initializes `mc_rng1` with its rank as the seed identifier, computes a local sum and participates in `MPI_Reduce` with `MPI_SUM`.

Listing 27: Local Monte Carlo computation, global sum and timing reductions in the MPI implementation

```

49  ...
50  call rng_seed(rank)
51
52  local_sum = 0.0_real64
53
54  call MPI_Barrier(MPI_COMM_WORLD, ierr)
55  start_time = MPI_Wtime()
56
57  do sample = 1_int64, n_local
58      call rng_uniform(x, y, z)
59
60      value = exp(-(x*x + y*y + z*z))
61      local_sum = local_sum + value
62  end do
63
64  call MPI_Reduce(local_sum, global_sum, 1, MPI_DOUBLE_PRECISION, &
65                MPI_SUM, 0, MPI_COMM_WORLD, ierr)
66
67  end_time = MPI_Wtime()
68  local_elapsed = end_time - start_time
69
70  call MPI_Reduce(local_elapsed, maximum_elapsed, 1, &
71                MPI_DOUBLE_PRECISION, MPI_MAX, 0, MPI_COMM_WORLD, ierr)
72  ...

```

At this point every rank executes the same code but on a different value of `n_local`. The RNG is initialized with the rank identifier, so each process maintains an independent RNG state. The local loop produces `local_sum`; the first `MPI_Reduce` then adds all local sums into `global_sum` on rank zero.

The initial `MPI_Barrier` ensures that all processes reach the timed region before the clock starts. Since individual ranks may complete at slightly different times, each process first computes its own `local_elapsed`. The second `MPI_Reduce`, this time using `MPI_MAX`, selects the largest elapsed time. This value represents the completion time of the parallel computation as a whole.

7.6 Hybrid MPI+OpenMP version

The hybrid program uses a two-level workload distribution. At the first level, MPI partitions the global sample count among the P MPI ranks using exactly the same quotient and remainder decomposition described in Section 7.5. At the second level, each rank creates an OpenMP team and distributes its local workload `n_local` among the T threads of that process using `schedule(static)`. In the experiments presented in this work all ranks are allocated on a single Lyra compute node, but each MPI process still owns an independent address space.

To safely support this architecture, the communication environment is initialized using `MPI_Init_thread` rather than the standard `MPI_Init`, alerting the underlying MPI library to the

presence of multi-threading.

Furthermore, the program explicitly requests the `MPI_THREAD_FUNNELED` support level. This establishes a strict execution contract: only the master thread of each MPI process is permitted to execute network calls (such as `MPI_Bcast` and `MPI_Reduce`), and it does so exclusively outside the OpenMP parallel regions. By guaranteeing that secondary OpenMP threads will only perform local Monte Carlo computations and never attempt simultaneous network communication, the MPI library can safely disable its costly internal thread-safety locks, thereby maximizing network throughput and eliminating unnecessary synchronization overhead.

Listing 28: MPI thread support request in the hybrid implementation.

```

26 ...
27     required_thread_level = MPI_THREAD_FUNNELED
28     call MPI_Init_thread(required_thread_level, provided_thread_level, ierr)
29
30     if (provided_thread_level < required_thread_level) then
31         write(error_unit, '(A)') &
32             'The MPI implementation does not provide MPI_THREAD_FUNNELED.'
33         call MPI_Abort(MPI_COMM_WORLD, 1, ierr)
34     end if
35 ...

```

Analyzing the code snippet above, the presence of the `if` condition serves as a critical safety validation for the runtime environment. The MPI standard defines thread support levels hierarchically, represented by strictly increasing integer values: `MPI_THREAD_SINGLE`, `FUNNELED`, `SERIALIZED`, and `MULTIPLE`. By checking whether the `provided_thread_level` is strictly less than the `required_thread_level`, the program ensures the system can safely support the hybrid architecture. If the system returns a lower level (e.g., `MPI_THREAD_SINGLE`), it indicates that the underlying MPI library lacks the necessary internal mechanisms to handle multi-threading, meaning that opening an OpenMP region would lead to race conditions or fatal memory corruption. Conversely, if the system provides a higher level than requested (such as `MPI_THREAD_MULTIPLE`), the execution can proceed safely without throwing an error, as the strict `FUNNELED` behavior implemented in our code is a naturally compliant subset of those broader system capabilities.

Listing 29: OpenMP work sharing among MPI Processes

```

35 ...
36     call MPI_Comm_rank(MPI_COMM_WORLD, rank, ierr)
37     call MPI_Comm_size(MPI_COMM_WORLD, nprocs, ierr)
38
39     n_total = 0_int64
40
41     if (rank == 0) then
42         call read_sample_count(n_total, io_status)
43
44         if (io_status /= 0) then
45             write(error_unit, '(A)') &
46                 'Input error: unable to read input/input.dat or invalid
47                 sample count.'
48             call MPI_Abort(MPI_COMM_WORLD, 1, ierr)
49         end if
50     end if
51
52     call MPI_Bcast(n_total, 1, MPI_INTEGER8, 0, MPI_COMM_WORLD, ierr)

```

```

53     base_points = n_total / int(nprocs, int64)
54     remainder = mod(n_total, int(nprocs, int64))
55
56     n_local = base_points
57     if (int(rank, int64) < remainder) then
58         n_local = n_local + 1_int64
59     end if
60
61     local_sum = 0.0_real64
62     actual_threads = 0
63
64     call MPI_Barrier(MPI_COMM_WORLD, ierr)
65     start_time = MPI_Wtime()
66     ...

```

After the MPI-level decomposition, every rank owns its local iteration count `n_local`. The second level of parallelism is then introduced by the following OpenMP region:

Listing 30: OpenMP work sharing inside each MPI rank of the Hybrid implementation.

```

66     ...
67     !$omp parallel default(none) &
68     !$omp& shared(n_local, actual_threads, rank) &
69     !$omp& private(thread_id, x, y, z, value) &
70     !$omp& reduction(+:local_sum)
71
72         thread_id = omp_get_thread_num()
73
74         call rng_seed(rank * 1000 + thread_id)
75
76         if (thread_id == 0) then
77             actual_threads = omp_get_num_threads()
78         end if
79
80         !$omp do schedule(static)
81         do sample = 1_int64, n_local
82             call rng_uniform(x, y, z)
83
84             value = exp(-(x*x + y*y + z*z))
85             local_sum = local_sum + value
86         end do
87         !$omp end do
88
89     !$omp end parallel
90     ...

```

The structure is deliberately similar to the pure OpenMP implementation, but there are three important differences:

- The loop limit is now `n_local` rather than the global sample count, because MPI has already assigned only part of the problem to each rank.
- The OpenMP reduction produces `local_sum`, which represents the contribution of one MPI process.
- Finally, the rank is shared by all threads of the process and is combined with `thread_id` when the RNG state is initialized. This gives every rank/thread pair a different seed identifier.

Once the OpenMP region has completed, one local sum remains for every MPI rank. These values are combined exactly once at the MPI level:

Listing 31: MPI aggregation and elapsed-time selection after the Hybrid OpenMP region.

```

90 ...
91     call MPI_Reduce(local_sum, global_sum, 1, MPI_DOUBLE_PRECISION, &
92                    MPI_SUM, 0, MPI_COMM_WORLD, ierr)
93
94     end_time = MPI_Wtime()
95     local_elapsed = end_time - start_time
96
97     call MPI_Reduce(local_elapsed, maximum_elapsed, 1, &
98                    MPI_DOUBLE_PRECISION, MPI_MAX, 0, MPI_COMM_WORLD, ierr)
99 ...

```

The complete reduction is therefore hierarchical. First, OpenMP combines the partial sums produced by the threads of one rank into `local_sum`. Then MPI combines the P rank-local values into `global_sum`. The final estimate is computed by rank zero as

$$\text{estimate} = \frac{\text{global_sum}}{\text{n_total}}.$$

The same two-level interpretation applies to the resource count: with P MPI processes and T OpenMP threads per process, the total number of processing elements used in the strong-scaling analysis is $P \times T$.

The Slurm job allocates these resources separately through `--ntasks` and `--cpus-per-task`, and the total number of processing elements used in the strong-scaling tables is $P \times T$.

8 Experimental Methodology

The four programs are executed with the same value of the global sample count. Each run writes its numerical estimate and measured execution time to the corresponding file in `output/`. Slurm messages are kept separately in `output-job/`. For the final performance analysis, the recorded times were collected into the file `results_N.10.8.xlsx` in order to compute speedup and efficiency for OpenMP, MPI and hybrid configurations.

8.1 Hardware and software environment

The experiments were submitted to the Lyra cluster through Slurm. The job scripts use the `dida-hpc` account, the `hpc-q` partition and a single allocated node. All tested configurations use at most 20 processing elements, consistently with the resource limit used in the project jobs.

Item	Value
Cluster	Lyra, UNIMORE
Scheduler	Slurm
Account and partition	dida-hpc, hpc-q
Nodes per run	1
Maximum processing elements tested	20
Memory requested by the jobs	40 GB
Compiler wrapper	Intel mpiifort
Optimization	-O2
OpenMP compiler option	-qopenmp for OpenMP and hybrid versions
MPI execution	mpiexec -n "\$SLURM_NTASKS"

Table 4: Software and job configuration used for the measurements.

The scripts load `slurm`, `oneapi/compiler`, `oneapi/mkl` and `oneapi/mpi`. The measurements therefore refer to the software environment created by these modules and to a single Slurm allocation on Lyra.

8.2 Numerical check before benchmarking

For the same value of N , all versions use the same unit-cube domain, integrand, double-precision representation and input file. The exact integral can be written as the product of three identical one-dimensional integrals:

$$I = \left(\int_0^1 e^{-x^2} dx \right)^3 = \left(\frac{\sqrt{\pi}}{2} \operatorname{erf}(1) \right)^3 \approx 0.4165383859. \quad (6)$$

Every executable writes an `Estimate` field, which can be checked against this reference value before its timing result is used. Bitwise equality between implementations is not expected. The final `mc_rng1` module uses different seed identifiers for threads and MPI ranks and also mixes in the system clock; in addition, parallel reductions can change the order of floating-point additions.

8.3 Timing methodology

The sequential version uses `CPU_TIME`; the OpenMP version uses `omp_get_wtime()`; and the MPI and hybrid versions use `MPI_Wtime()`. The timed regions implemented in the final source files are summarized below.

Version	Timed region
Sequential	Sampling loop only; input, RNG initialization, estimate formation and output are excluded.
OpenMP	From immediately before the parallel region to immediately after it; includes team creation, per-thread RNG initialization, the loop, reduction and implicit synchronization.
MPI	From after rank-local RNG initialization and the MPI barrier to after the global sum reduction; the reported time is the maximum elapsed time across ranks.
Hybrid	From after the MPI barrier to after the global sum reduction; includes OpenMP team creation, per-thread RNG initialization, the local loop, OpenMP reduction and MPI sum reduction. The maximum time across ranks is reported.

Table 5: Timed regions implemented in the final repository.

8.4 Strong-scaling experiments

Strong scaling keeps the global problem size fixed while the available parallelism is increased. For the final measurements the input file contains

$$N = 10^8 \quad (7)$$

Monte Carlo samples. The same sample count is used for every configuration. Only the Slurm resource directives are changed between runs.

The OpenMP experiment varies the number of threads by changing `--cpus-per-task`. The MPI experiment varies the number of processes by changing `--ntasks`. The hybrid experiment changes both values. For a hybrid configuration, the total number of processing elements is:

$$p_{\text{total}} = P \times T, \quad (8)$$

where P is the number of MPI processes and T is the number of OpenMP threads per process.

The tested configurations are:

Implementation	Configurations
OpenMP	1, 2, 4, 8, 16, 20 threads
MPI	1, 2, 4, 8, 16, 20 processes
Hybrid	$(P, T) = (1, 1), (1, 2), (2, 2), (2, 4), (4, 4), (4, 5)$

Table 6: Configurations used for the strong-scaling experiments.

All hybrid configurations satisfy $P \times T \leq 20$.

8.5 Performance metrics

For the strong-scaling analysis, each implementation uses its own single resource configuration as the reference. The OpenMP baseline is the execution of one thread, the MPI baseline is the

execution of one process, and the hybrid baseline is the execution with one MPI process and one OpenMP thread.

The strong-scaling speedups are

$$S_{\text{OMP}}(p) = \frac{T_{\text{OMP}}(1)}{T_{\text{OMP}}(p)}, \quad S_{\text{MPI}}(p) = \frac{T_{\text{MPI}}(1)}{T_{\text{MPI}}(p)}. \quad (9)$$

where p is the number of OpenMP threads or MPI processes, depending on the implementation.

For the hybrid implementation,

$$S_{\text{HYB}}(P, T) = \frac{T_{\text{HYB}}(1, 1)}{T_{\text{HYB}}(P, T)}. \quad (10)$$

The corresponding efficiencies are:

$$E_{\text{OMP}}(p) = \frac{S_{\text{OMP}}(p)}{p}, \quad E_{\text{MPI}}(p) = \frac{S_{\text{MPI}}(p)}{p}, \quad (11)$$

and

$$E_{\text{HYB}}(P, T) = \frac{S_{\text{HYB}}(P, T)}{P \times T}. \quad (12)$$

The sequential execution is used only as an additional one-core reference for the four implementations. It is not used as the speedup baseline for OpenMP, MPI or hybrid scaling.

9 Results

The final measurements use a fixed global sample count of $N = 10^8$. Speedup and efficiency are calculated with the single resource execution of the same implementation as the baseline.

9.1 Single-resource comparison

Before considering strong scaling, the one resource executions give a useful reference for the overhead introduced by the different programming models. The sequential program requires 1.798848 s. The one-thread OpenMP and the $(P, T) = (1, 1)$ hybrid executions require 2.651437 s and 2.753376 s, respectively, while the one-process MPI execution requires 7.779876 s.

Version	Configuration	Time [s]
Sequential	1 process	1.798848
OpenMP	1 thread	2.651437
MPI	1 process	7.779876
Hybrid	1 MPI process $\times$ 1 thread	2.753376

Table 7: Single-resource execution-time comparison for $N = 10^8$.

The programs also write the Monte Carlo estimate to their individual output files. The analytical value in equation 6 is approximately 0.4165383859 and provides the numerical reference for checking those estimates.

9.2 OpenMP, MPI and hybrid strong-scaling results

Threads	Time [s]	Speedup	Efficiency
1	2.651437	1.000	100.00%
2	1.541203	1.720	86.02%
4	0.934554	2.837	70.93%
8	0.541902	4.893	61.16%
16	0.294329	9.008	56.30%
20	0.255757	10.367	51.84%

Table 8: OpenMP strong-scaling results for $N = 10^8$.

The OpenMP execution time decreases monotonically as the thread count is increased. At 20 threads the measured speedup is 10.367, corresponding to an efficiency of 51.84%. The efficiency decreases progressively because the parallel overhead becomes a larger fraction of the remaining execution time, but no reversal of the execution-time trend is observed in the tested range.

MPI processes	Time [s]	Speedup	Efficiency
1	7.780000	1.000	100.00%
2	4.276176	1.819	90.97%
4	2.197878	3.540	88.49%
8	1.244304	6.252	78.16%
16	0.717255	10.847	67.79%
20	0.584758	13.305	66.52%

Table 9: MPI strong-scaling results for $N = 10^8$.

The MPI implementation shows a regular strong-scaling behaviour over the tested process counts. Execution time decreases from 7.780 s with one process to 0.584758 s with 20 processes. The corresponding speedup reaches 13.305, with an efficiency of 66.52%. As expected in a strong-scaling experiment, efficiency decreases as the number of processes increases because the amount of computation assigned to each rank becomes smaller while MPI runtime, synchronization and collective reduction costs remain present.

MPI processes	Threads/process	Total elements	Time [s]	Speedup	Efficiency
1	1	1	2.753376	1.000	100.00%
1	2	2	1.498038	1.838	91.90%
2	2	4	0.868829	3.169	79.23%
2	4	8	0.483681	5.693	71.16%
4	4	16	0.260559	10.567	66.04%
4	5	20	0.234514	11.741	58.70%

Table 10: Hybrid MPI+OpenMP strong-scaling results for $N = 10^8$.

The hybrid implementation also decreases its execution time monotonically. At 20 total processing elements, configured as four MPI processes with five OpenMP threads each, the measured speedup is 11.741 and the efficiency is 58.70%. For the same total numbers of processing elements used in the tables, the hybrid timings are lower than the corresponding OpenMP timings from two processing elements onward. At 20 processing elements the hybrid run requires 0.234514 s compared with 0.255757 s for OpenMP.

9.3 Strong-scaling figures

The following figures report the strong-scaling speedup for the three parallel implementations. Each curve is normalized to the corresponding single resource execution: one thread for OpenMP, one process for MPI and one MPI process with one OpenMP thread for the hybrid version. The ideal linear speedup is included in each graph as a reference.

9.3.1 OpenMP Results

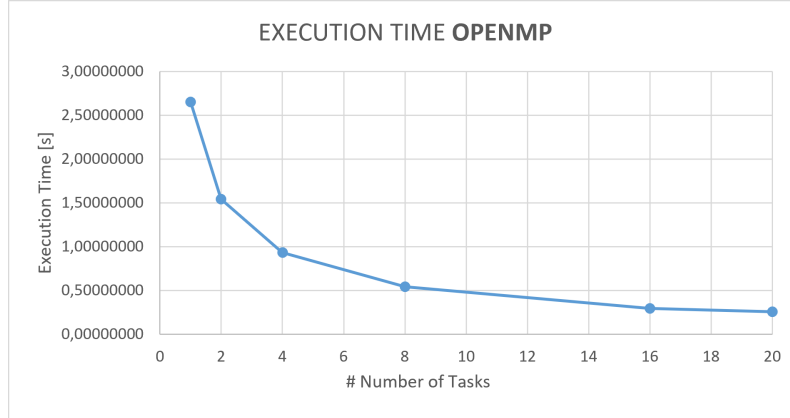


Figure 1: OpenMP execution time as a function of the number of threads for $N = 10^8$.

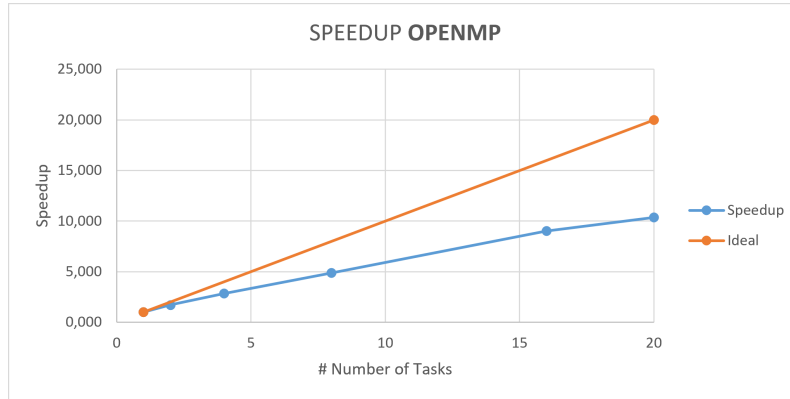


Figure 2: OpenMP speedup as a function of the number of threads for $N = 10^8$.

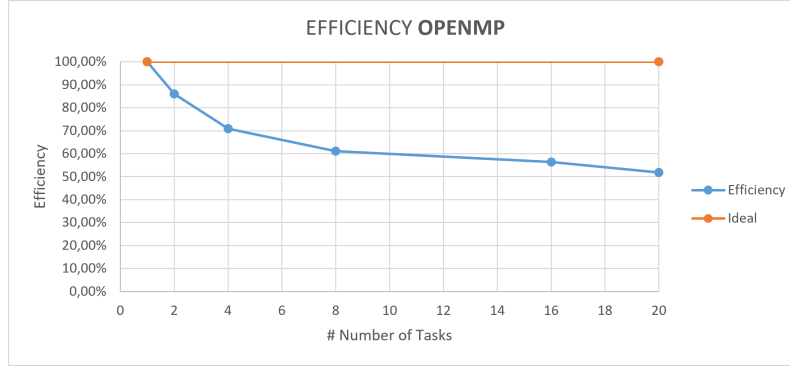


Figure 3: OpenMP parallel efficiency as a function of the number of threads for $N = 10^8$.

9.3.2 MPI Results

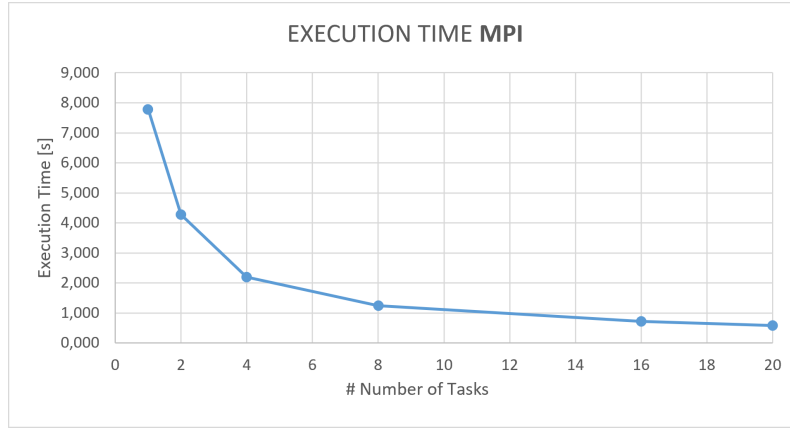


Figure 4: MPI execution time as a function of the number of processes for $N = 10^8$.

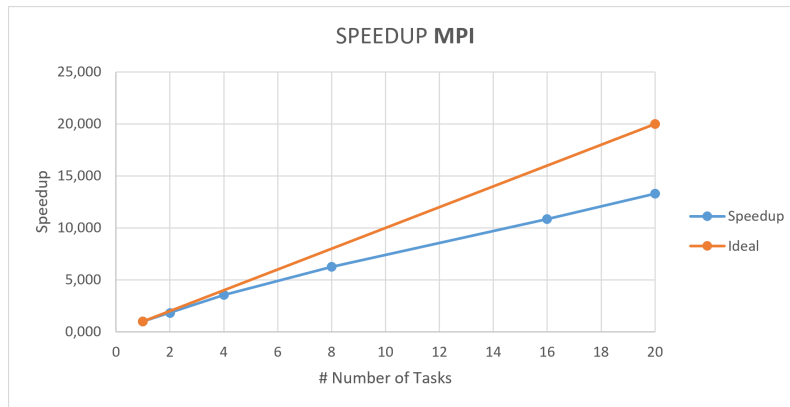


Figure 5: MPI speedup as a function of the number of processes for $N = 10^8$.

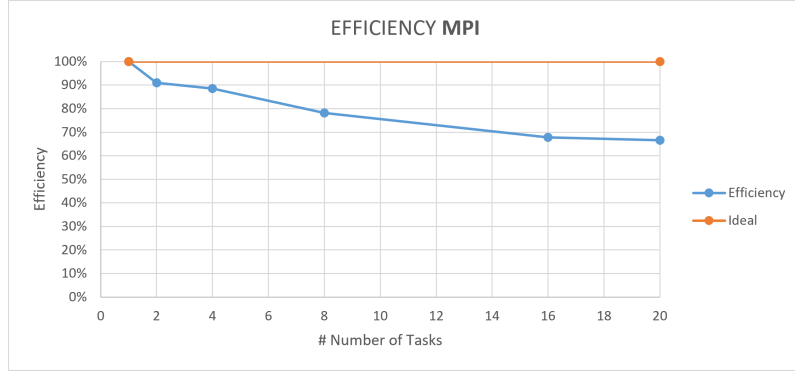


Figure 6: MPI parallel efficiency as a function of the number of processes for $N = 10^8$.

9.3.3 Hybrid Results

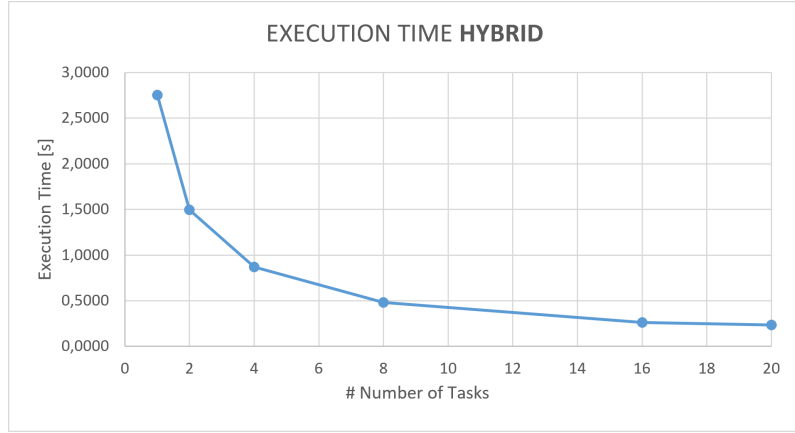


Figure 7: Hybrid execution time as a function of the total number of processing elements for $N = 10^8$.

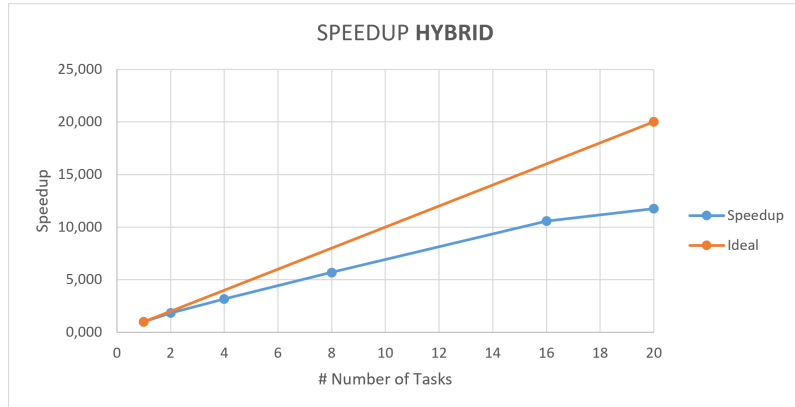


Figure 8: Hybrid speedup as a function of the total number of processing elements for $N = 10^8$.

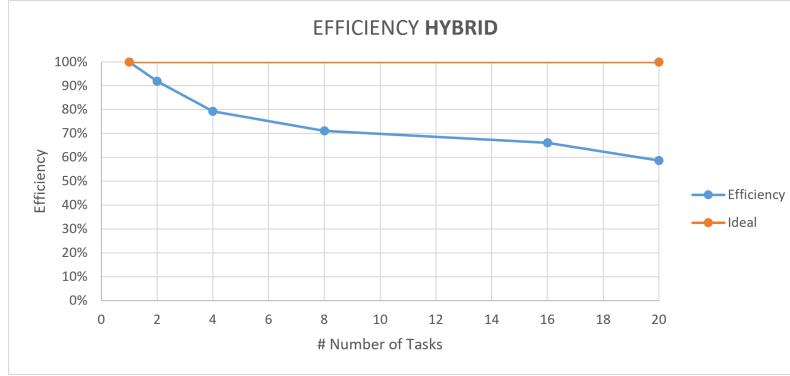


Figure 9: Hybrid parallel efficiency as a function of the total number of processing elements for $N = 10^8$.

10 Discussion

The final measurements show a regular reduction in execution time for all three parallel implementations as the number of processing elements is increased. OpenMP, MPI and Hybrid all exhibit positive strong-scaling behaviour, although with different efficiency trends.

Table 11 compares the three implementations at 20 total processing elements for $N = 10^8$. Each speedup uses the single-resource execution of the same implementation as its baseline; efficiency is calculated as $S/20$ and expressed as a percentage.

Version	Configuration	Speedup	Efficiency
OpenMP	20 threads	10.367	51.84%
MPI	20 MPI processes	13.305	66.52%
Hybrid	4 MPI processes $\times$ 5 threads/process	11.741	58.70%

Table 11: Speedup and efficiency at 20 total processing elements for $N = 10^8$.

For OpenMP, the speedup increases from 1.720 with two threads to 10.367 with 20 threads. The corresponding efficiency decreases from 86.02% to 51.84%. This is a conventional strong-scaling trend: the fixed amount of Monte Carlo work is divided among more threads, while the costs of creating the team, initializing per-thread RNG states, synchronizing the threads and performing the reduction do not decrease in the same proportion. Static scheduling is well suited to this loop because all iterations perform essentially the same amount of work.

The Hybrid implementation also shows regular strong scaling. The (1, 2) configuration reaches a speedup of 1.838, and the speedup increases to 11.741 for $(P, T) = (4, 5)$. Its efficiency at 20 processing elements is 58.70%, compared with 51.84% for pure OpenMP. In absolute terms, the final Hybrid configuration requires 0.234514 s, compared with 0.255757 s for the 20-thread OpenMP execution. This does not establish that the Hybrid model is always better; it only shows that the selected process/thread decomposition works well for this problem and this single-node configuration.

The MPI implementation also exhibits a regular strong-scaling trend. The execution time decreases from 7.780 s with one process to 4.276176 s with two processes, and continues to decrease as additional ranks are introduced, reaching 0.584758 s with 20 processes. The corresponding speedup increases to 13.305, while the efficiency decreases from 90.97% with two processes to

66.52% with 20 processes. This reduction in efficiency is expected in strong scaling because the local workload becomes progressively smaller, whereas process-management, synchronization and collective-reduction overheads do not decrease proportionally. Among the three parallel implementations, MPI therefore reaches the highest relative speedup and efficiency at 20 processing elements, although its absolute execution time is higher than the corresponding OpenMP and Hybrid measurements. This distinction is important because each speedup is normalized to the single-resource execution of the same implementation.

The change from `mc_rng` to `mc_rng1` is important when interpreting the OpenMP and Hybrid results. In the final code, random number state is explicit and `threadprivate`, so each OpenMP thread owns its own RNG state instead of relying on the hidden state managed by the intrinsic Fortran random number generator. This removes the contention and race risk identified during the initial multithreaded tests and allows random number generation to proceed independently inside each thread. MPI ranks also have independent states because they execute in separate address spaces.

The numerical estimates from different executions are not expected to be bitwise identical. The RNG initialization includes the system clock and uses different seed identifiers for threads and ranks. In addition, OpenMP and MPI reductions can change the order of floating-point additions. Numerical correctness should therefore be assessed by comparison with the analytical reference in equation 6, not by requiring identical output across different configurations.

The conclusions are limited to this independent-sample Monte Carlo problem, the tested value $N = 10^8$, the selected process/thread configurations and the single-node Lyra environment. The same performance behaviour cannot be assumed for communication-intensive, memory-bound or irregular algorithms.

11 Conclusions

The project implemented the same three-dimensional Monte Carlo integral in four forms: sequential, OpenMP, MPI and Hybrid MPI+OpenMP. Using one common `mc_rng1` module made the comparison more consistent and, more importantly, removed the dependence of the threaded versions on the hidden state of the Fortran intrinsic random-number generator. The final RNG keeps an explicit state for each OpenMP thread and an independent state for each MPI process.

The OpenMP implementation shows a regular strong-scaling trend for the tested sample count $N = 10^8$. Its execution time decreases from 2.651437 s with one thread to 0.255757 s with 20 threads, corresponding to a speedup of 10.367 and an efficiency of 51.84%. The loss of efficiency at higher thread counts is consistent with the increasing relative weight of parallel-region, synchronization and reduction overheads.

The MPI implementation also achieves consistent strong scaling. Its execution time decreases from 7.780 s with one process to 0.584758 s with 20 processes, corresponding to a speedup of 13.305 and an efficiency of 66.52%. Although efficiency decreases as the process count grows, the measured trend remains regular over the complete range of tested configurations. At 20 processing elements, MPI provides the highest relative speedup and efficiency among the three parallel implementations.

The Hybrid implementation also scales regularly. With four MPI processes and five OpenMP threads per process it reaches 0.234514 s, a speedup of 11.741 and an efficiency of 58.70%. Although its relative speedup is lower than the MPI value, this configuration gives the lowest absolute execution time among the parallel measurements at 20 processing elements. The result shows that, for this single-node Monte Carlo workload, a mixed process/thread decomposition can use the avail-

able resources effectively.

Table 12 summarizes the execution times at the same total allocation of 20 processing elements for $N = 10^8$.

Version	Configuration	Execution time [s]
OpenMP	20 threads	0.255757
MPI	20 MPI processes	0.584758
Hybrid	4 MPI processes $\times$ 5 threads/process	0.234514

Table 12: Execution times at 20 total processing elements for $N = 10^8$.

Overall, the project confirms that the Monte Carlo sampling loop is naturally parallel because samples are independent and the only global operation is the reduction of partial sums. It also shows that implementation details outside the mathematical algorithm matter: random number state, thread affinity, MPI process placement and the exact timed region can materially affect the measured scaling. The reported conclusions therefore apply to the current code, the Lyra single node jobs and the configurations tested here, rather than to OpenMP or MPI in general.

References

- Topic-to-source map.** OpenMP execution, data sharing, reductions, thread-private state, scheduling and affinity: [1]; MPI, Monte Carlo integration and strong-scaling methodology covered by the course: [2]; Fortran intrinsic RNG thread safety: [3]; the project's OpenMP RNG issue, dedicated parallel-generator rationale and stream-quality limitation: [4]; per-thread state semantics: [1]; LCG recurrence parameters: [5]; false sharing: [6]; Intel Fortran OpenMP compiler option: [7].
- [1] OpenMP Architecture Review Board, *OpenMP API Specification, Version 5.2*, November 2021. Supports the OpenMP execution model, directives, data-sharing clauses, reductions, `threadprivate`, work-sharing schedules, runtime routines and environment variables. Available at: <https://www.openmp.org/spec-html/5.2/openmp.html>.
 - [2] *Course materials for High Performance Computing*, provided by the course instructor, University of Modena and Reggio Emilia, n.d. Supports the report's discussion of MPI processes and collective operations, Monte Carlo integration, and strong-scaling methodology.
 - [3] Intel Corporation, *RANDOM_NUMBER*, Intel Fortran Compiler Developer Guide and Reference, 2023. Documents the intrinsic's thread safety; this does not guarantee high throughput. Available at: <https://www.intel.com/content/www/us/en/docs/fortran-compiler/developer-guide-reference/2023-1/random-number.html>.
 - [4] GNU Project, *RANDOM_NUMBER — Pseudo-random number*, GNU Fortran Manual, Version 5.5.0; P. L'Ecuyer et al., *Multiple Streams with Recurrence-Based, Counter-Based, and Splittable Random Number Generators*, GERAD G-2021-38, 2021. These sources contextualize the project's OpenMP RNG change: GNU recommends considering a dedicated generator for RNG-intensive OpenMP code, and L'Ecuyer et al. discuss the design and testing of parallel streams. They do not identify the cause of the slowdown measured with Intel on Lyra. Available at: https://gcc.gnu.org/onlinedocs/gcc-5.5.0/gfortran/RANDOM_005fNUMBER.html; <https://www.gerad.ca/en/papers/G-2021-38>.
 - [5] The Open Group, *rand, rand_r, srand — pseudo-random number generator*, POSIX.1 example implementation. The illustrative recurrence uses the multiplier and increment used in this project's LCG; it is cited for parameter provenance, not as a quality recommendation. Available at: https://pubs.opengroup.org/onlinepubs/7908799/xsh/rand_r.html.
 - [6] Intel Corporation, *False Sharing*, Intel Performance Skills, Performance Patterns. Supports the description of cache-line contention caused by independent per-thread writes to data sharing a cache line. Available at: <https://timmastny.com/blog/false-sharing/>.
 - [7] Intel Corporation, *Add OpenMP Support*, Intel Fortran Compiler Classic and Intel Fortran Compiler Developer Guide and Reference, 2024. Documents the `-qopenmp` compiler option used in the project. Available at: <https://www.intel.com/content/www/us/en/docs/fortran-compiler/developer-guide-reference/2024-2/add-openmp-support.html>.